

An IoT-based Power Management System for EV Chargers

Vimalarani G
Dept. of ECE
Hindustan Institute of Technology
and Science
Chennai, Tamil Nadu, India
gvrani@hindustanuniv.ac.in

Kavitha B C
Dept. of ECE
Hindustan Institute of Technology
and Science
Chennai, Tamil Nadu, India
bckavitha@hindustanuniv.ac.in

Shaik Nihal
Dept. of ECE,
Hindustan Institute of Technology
and Science
Chennai, Tamil Nadu, India
19121001@student.hindustanuniv.ac.in

K Rakesh
Dept. of ECE
Hindustan Institute of Technology
and Science
Chennai, Tamil Nadu, India
19121016@student.hindustanuniv.ac.in

ABSTRACT --- In the fast developing world, EV plays a major role. Less resource utilization, cost efficient, less maintenance, long durability are some factors which make people to shift towards Electrical vehicles. Even though a huge number of people are now showing interest towards Electrical Vehicles, there are only limited power stations available and most of the time the coordinates of the power stations are unknown to EV users. Hence, a power management system is required wherein the EV users can access the power stations easily and increase the number of Electrical Vehicles stations. As in other side people are now shifting towards Internet of Things and Cloud technology to make their life better and to access things easily. Using EV's as their source of traveling, it reduces cost and enhances many other factors. Since both IoT and EV are becoming the fast adapting technologies, these two technologies can be merged in future. In the proposed research work, both IoT and EV's plays a main role. Here, a prototype to easily access charging station was developed and then integrated with Web Technology to show where and when the user can access the charging stations. The proposed idea would help in increase the number of charging stations, which indirectly increase the usage of EVs.

Keywords— *Electric Vehicle (EV) chargers, Current Transformer (CT) sensor, Web Application, Cloud Interfaces.*

I. INTRODUCTION

In this modern world, everyone is adapting to use EV Vehicles to reduce the fuel consumption and cost. The users of EV vehicles are provided with an EV charger to charge their EV vehicles. But users are unable to find out

the power source when there are traveling a long distance which make one of the biggest drawbacks in EV system. So there need to be a system where normal people can provide their energy source to EV chargers and the EV user has access to the location of the EV power stations with the help of Web Application.

Life in the modern world is getting better day by day with new innovations and developments in multiple fields, and one of the major factors is cost efficiency. Nowadays everyone is bother about how much amount they have spent on their day-to-day life activities. Recently there many researches are carried to calculate different aspects of individual human activities using different technology.

This is one of the methodologies, where EV users can access the power stations and can track the amount spend on their EV charging. In the proposed article a current transformer sensor (SCT-013-030 non-invasive AC current sensor) is used to track the amount of power consumed by their EV's with the help of Arduino Uno and Arduino Uno is integrated with Raspberry Pi to share the data over the cloud. So, that user can track the on a webpage via getting information through cloud.

As the title describes the IoT based power management, here the webpage as a user interface provides system a power monitoring system with a control EV charging process. Initiate of the charging sessions, viewing the resultant cost of the power consumed by their EV's from any edge of the world make a system IoT integrated system.

II. LITERATURE REVIEW

Altaf Hamed et al. [1], the authors have designed a smart plug using ENC28J60 ethernet module and current transformer sensor. The author designed a environment in a such a way that the current transformer sensor extracts the current details with the help of Arduino and Arduino

in returns send data to ENC28J60 ethernet module. The author developed an android application to show the power consumption.

T. Juhana et al. [2], the author created an environment to display the voltage and power consumption using Bluno and current transformer sensor. Current transformer senses the current consumption and sends data to Bluno. Bluno uses Bluetooth connectivity to share the data to server where power and voltage consumption are showed.

S. Ren et al. [3], the author discussed the methodology to display the voltage and power consumption via LCCR meter and Digital Wattmeter. Current consumption values are generated by current transformer sensor and return values to the converter board to convert the values in terms of voltage and power. With the help of these values waveforms and consumptions are showed

S. E. princy et al. [4], the author created an environment to transfer data between Arduino Uno to Raspberry Pi and to cloud server. A serial communication between Arduino and Raspberry Pi was implemented to share the data. Raspberry Pi is connected to OwnCloud cloud server to store the data generated by the Arduino

III. METHODOLOGY

As the proposed ideology is an integration of both IoT and cloud technology, the methodology has two different stages. First one is Hardware setup where the current consumption is tracked by CT sensor and a serial communication between the Arduino Uno and Raspberry Pi. Second one is software setup where our Raspberry pi is connected to Internet to share the data over cloud which in returns given as input to user interface webpage.

As in the Hardware setup there are multiple steps to carried out. At first need to create a environment to track the current consumption. This process is carried out by current transformer sensor. The CT sensor used in this is SCT-013-030 non-invasive AC current sensor which is shown in fig.1. In a real time, basis current values are sensed and are transmitted to the server via Raspberry Pi which is connected to Arduino Uno. Maximum current of 30A can be sensed by the SCT-013-030 split core current transformer sensor and produce a output of 1V. the output should be compatible with the Arduino Uno. To make output compatible need to ratify the output signal with the help of a diode bridge and need to amplify it. The fig. 2 shows the connection between the CT sensor and Arduino Uno.

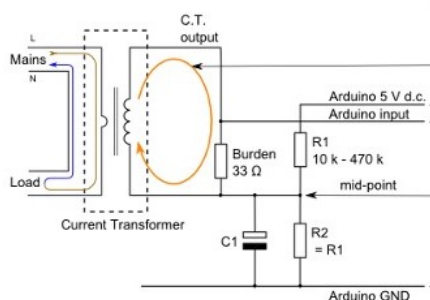


Figure 1. CT sensor to Arduino Uno connection

The fig. 1 shows the circuit connect between the CT sensor and Arduino Uno. The current sensor senses an AC voltage signal which is proportional to the current varies from 2.5V to 5V. In order to find out relationship between V_{rms} and I_{rms} first need to find out I_{ratio} which is given by

$$I_{ratio} = \frac{CT_TURNS}{CR_{Burden\ Resistor} * 5 * (Number\ of\ samples)}$$

The relationship between V_{rms} and I_{rms} is given by

$$I_{rms} = I_{ratio} \sqrt{V_{rms}}$$

Whew CT_TURNS represents the number of turns of CT coil's secondary coil which has a value of 1800 and $CT_{Burden\ resistor}$ has value of 620 Ohm as the initial burden resistor of SCT-013-030. As for the number of samples it is 1024 because here Arduino Uno is being used which an ADC of 10bit. So, number of samples = $1023(2^{10})$. Finally, the power consumption P is calculated by the formula

$$P = 230 * I_{rms}$$

Next, a serial communication is established between Raspberry Pi and Arduino Uno to transfer the cost generated for every second. Based on the instances received from the cloud, Raspberry pi will sum up the cost and generate the total cost within the charging period. With the closing instances Raspberry Pi will send the Total Cost data to Web Application with Cloud as the server interface. The fig. 2 describes the working principle behind Raspberry Pi connection and system establishment. The system generates total cost using cost generated per second when user clicks start instance and next when the user clicks Stop instance the system sends the data to the Web Page. The instances operations are carried out using Cloud.

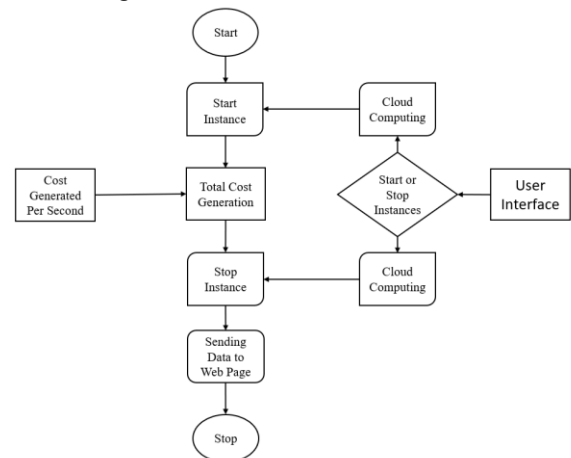


Figure 2. Raspberry Pi interface with cloud

The fig. 2 shows the workflow between raspberry pi and cloud like how the charging session start and stops, how total cost is generated and when. While coming to software setup first, the power consumption values are read by the Arduino Uno and Arduino Uno is coded in such a way that it reads the power consumption values convert them to necessary current values to calculate the

cost values. Next these values are shared to Raspberry Pi in serial communication.

Raspberry starts reading these values when there is Start instance from the user interface till the Stop interface. Raspberry Pi sums up all the cost generated per every second for the charging time period. When there is Stop instance Raspberry Pi sends the total cost generated to the User Interface Web page.

As AWS is being used cloud services to create instances. To create an instance in AWS cloud, first need to log in to our AWS console using our credentials. Once logged in, select the EC2 service from the list of available services to start creating a new instance. Choose an Amazon Machine Image (AMI) that contains the software and configuration required to launch and run the instance. Then, need to select the instance type that best fits the system needs in terms of processing power, memory, storage, and other resources.

After choosing our AMI and instance type, next need to configure the networking settings for the system instance, including the virtual private cloud (VPC), subnet, security group, and any other network-related settings. Also need to choose the storage options for the created instance, including the type and size of the root volume and any additional volumes may need.

Once the instances settings have been configured, can be reviewed the configuration and launch the instance. During the launch process, there may need of configure additional settings or options that are required for the specific use case, such as user data, tags, and monitoring settings. Once the instance is launched, can be connected to it using a remote desktop connection or a command-line interface, depending on the operating system and software that have been installed on the instance.

The website methodology involves in creating one instance with two functions one for start and one for stop which determines our charging time period. These functions can be triggered by using POST request in backend JavaScript. The User Interface Web page has some buttons to POST the request.

Next a Web Application has different Web pages based on who's using it. It has different Web pages for the EV users and different Web pages for the source providers. Let Considering the user of Web Application is EV users, at first EV users has been provided with Login Web page where all the details of EV users are getting stored for the future usage. Next the EV users are redirect to another Web page where they can see list of EV charging stations available. On selecting one they are again redirected to final Web page where users are provided with three buttons named as "START", "STOP" and "CALCULATE".

When the user clicks "START" button a GET request been fetched which make start function from the instance in the AWS environment that was created. When the user clicks "STOP" button a GET request been fetched which make stop function from the instance in the AWS

environment that was created. Finally, when user clicks "CALCULATE" button it will display the total cost, user should pay to source providers for providing power source.

The fig. 3 explains about the workflow in Web Application of the proposed system. As seen in figure Web Application is divided into two parts based on EV user or Client. Here Clients are the providers of power source. When someone select Client's option, they will redirect to a separated Web page where the client credentials are taken from them. The credentials such as name, location, availability is stored and shown in EV user Web page. Next when EV user uses the Web Application, they are redirected to User Registration pages. After registration, the availability of EV charging stations is shown to EV users. When an EV user selected the available EV charging station, they will be redirected to Main User Interface Web page. In this page EV users perform the necessary actions and generate the Total cost for the power consumption and will pay to the power source provider.

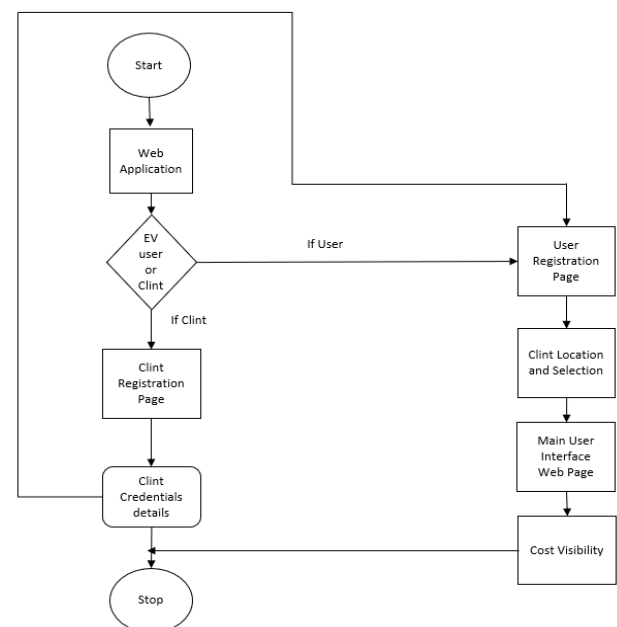


Figure 3. Website Workflow

The fig. 3 explains how the designed website works and talks about the conditions which help what pages should load. The overall implementation of the follow's as, at first the CT sensor is connected to live wire and with the help of breadboard the data generated is given to the Arduino Uno. The Arduino Uno reads the data convert the data into the cost of power consumption per second. Next the Raspberry Pi is connected to Arduino in serial communication. Raspberry Pi is coded in such way that it reads the values coming from serial communication only when the server gives start instances. It will read the until the server gives stop instances. The start and stop instances are provided by the backend GET request via AWS cloud. In AWS we created two instances and Raspberry Pi is connect to the user AWS account.

When the Cloud provokes Start instance the Raspberry will start reading the data from Arduino Uno in serial communication. When Cloud provokes Stop instances the Raspberry Pi will stop reading the data and sums up the cost generated per every second to find the total cost for charging period nothing but time interval between Start and Stop instances.

After generating Total cost will send this data to backend and in backed it will get stored. Using Frontend HTML and CSS tags, the Total cost generated will be displayed to the EV users which they need to pay to the power source providers.

The proposed ideology is carried out in a step-by-step manner and at each stage the output results are verified. In initial stage CT sensor values are just printed in Arduino IDE Serial Monitor. Similarly at every stage the dummy values are generated, and overall result output is extracted from the integrations and implementations of this test cases.

IV. RESULTS AND DISCUSSIONS

The proposed work has been carried out and the result in each stage is generated. Based on the results generated in each stage, final output required for the article is reached.

In initial stage, RMS values of the current are printed which are generated by the CT sensor in Arduino IDE to check whether the CT is working properly and to check whether Arduino Uno was able to read the values generated by the CT sensor.

After getting current and power values with modifications in the code the cost of power consumed per second can be generated. These values are printed in serial monitor of the Arduino IDE. While calculating cost many factors need to consider. To calculate the cost, the user should know the unit charges in local, one can get these details from the current department. And considering power source providers 1 or 2 rupees more can be added as a profit for them. Before generating cost, energy is calculated using formula $E = Pt$, where P is power and t is time period. Here the system is used to calculate for only one second that makes our Energy equals to Power. The cost generated here will be in form of WSec unit, but for the system it needed to convert it into KWhr as unit charges are considered in KWhr. After all necessary conversions the cost per every second is calculated and printed in serial monitor of Arduino Uno.

Using serial communication, the system is going establish a connection between the Arduino Uno and Raspberry Pi. Make sure both devices are switched down before attempting to link an Arduino Uno and a Raspberry Pi over serial. After establishment, in Arduino IDE a function is written in which will send the data to connected device in serial communication. Raspberry pi will starts reading the data only when there are instances from cloud. But for testing purpose a environment is created to read the current and power values which are generated Arduino test case code. Later after creating instance, setting up the cloud and Web Application, can

rewrite the Raspberry Pi to only read the values from the Start of the instances to the Stop functions of the Instance. There are various processes involved in setting up an AWS cloud instance. First register for an AWS account before logging into the AWS Management Console. Once in the console, start building an instance by choosing the "EC2" service. Select the location and availability zone where wishes to launch the created instance in the EC2 dashboard. Choose the AMI (Amazon Machine Image), a pre-configured virtual machine image that includes the required software and configurations to launch an instance, after choosing the region and zone. The instance type chosen next will influence the current instance's processing speed, memory size, and storage capacity. The instance can then be started and accessed remotely via SSH or other protocols. With installing the software and begin utilising the instance for the particular needs, such as hosting a website, managing a database, or processing data, after it is up and running.

Configure our instances to tell Raspberry Pi to Start and Stop the reading from the Arduino Uno. This instance can be started using GET request from the backend Web Application.

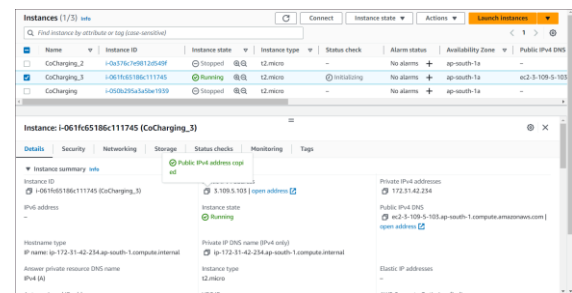


Figure 4. AWS Cloud Interface

The fig 4 shows the system cloud interfaces where one can see the Public IPv4 address to access the instance created. Considering the above figure CoCharging_3 instance is created and used here.

The output terminal of the raspberry pi which tells events that ours. By observing the above image one can see that one GET / start protocol and one GET / stop protocol are being used. These protocols are being implemented using the public IP address.

As proposed in the methodology, multiple Web pages are created for multiple purposes.

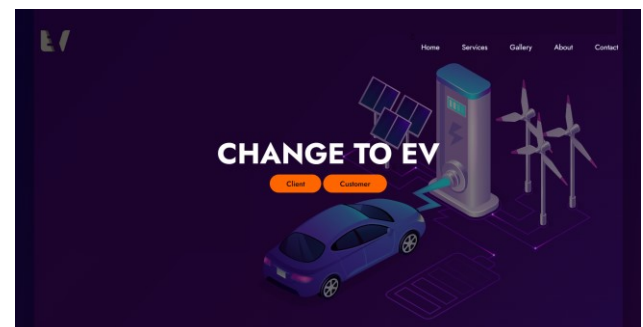


Figure 5. Common Web Page

The fig. 5 is the web page where both EV User and source providers. As seen, there are two buttons named as Client and Customer in the Web page. When the user clicks Client button, the user will be redirected to Client based Web pages and similarly when the user clicks Customer, user will redirect to EV users-based Web pages.

Let the Web Application user be Client who provides power source to the EV charger. So, when the user clicks Client button user will redirect to the Client based registration page where user is given to fill the form. The Client credentials are taken in form of forms and saved in the backend. These credentials are again used in Customer Web pages to show where charging stations are available near to them. When the user clicks the submitted button user will be redirected to the acknowledgement page.

The “Anchor” tags are used to redirect between the multiple HTML Web pages and the “OnClick” functionality of the button tag is used to trigger and redirected to other Web page via link present inside the anchor tag. The figure 4.3 is the Web page when the user submitted the form. This Web pages the acknowledgement that user submitted form is saved and the device user purchased from the company is ready to provide the service.

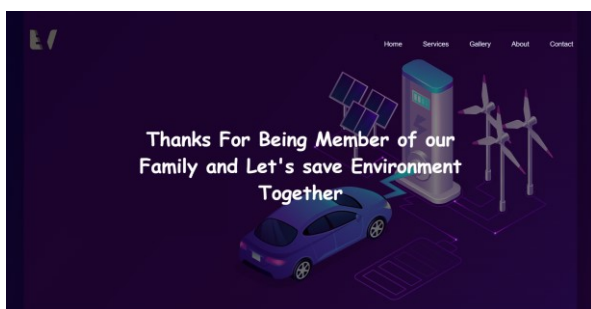


Figure 6. Client Acknowledgement Web Page

The Web pages also consists of some more additional buttons as seen in the top most right corner of the Web page. The Home button will redirect to the home page which is nothing but our common Web Page. Next Services button, when user clicks it, the pages will scroll down and show the respective content and the same functionality is implemented with all other buttons Service, Gallery, About and Contact.

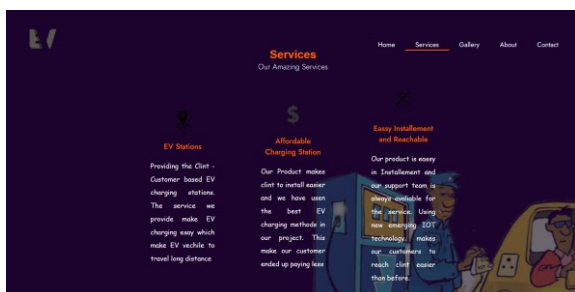


Figure 7. Additional Divisions in Web Application

The fig. 7 shows the service provided by the company for both Client and customer. This web divisions tells how the company works and what are the benefits of using EV transportation in day-to-day life. With this additional web

divisions, the user will have knowledge how much EV sustains the environment. Even though this is not related, it gives a well awareness to the people about the impact of EV's as the transportation and also tells how cheap using the EV's.

In the common Web Page when the user selects the customer, the user will redirect to the EV user-based Web pages. When the user clicks the Customer button user will redirect to Customer based Registration Page. The data taken from EV users are name, Email, Contact, Car/Bike EV and Vehicle number. The is stored in the JSON (Java Script Object Notation) with the help of backed in the Data Base. This data can be accessible by the Analytical companies to perform the overall analysis of the EV users such as how many cars/bikes are using this product, what's the cost spent by the EV users on average and many more.

Next when the user submits the credentials, the user will be redirected to another Web page where user will be shown all the available EV. User would be able to select the EV charging Station. After selection user will be redirected to main user interface Web page which contains multiple buttons to perform the necessary actions.

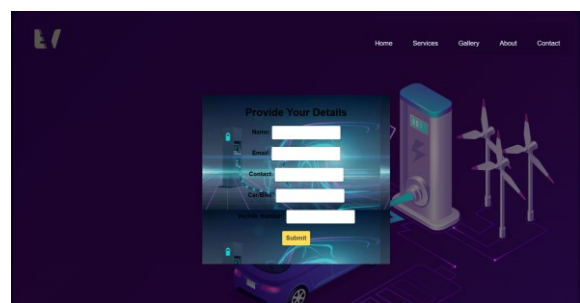


Figure 8. Customer Registration Web Page

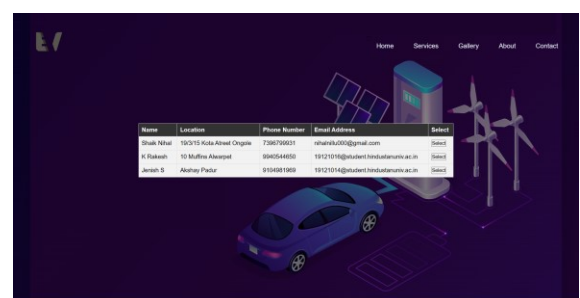


Figure 9. Charging Station Locations

In fig 8, the user is provided with the Client credentials details and location of the EV charging stations. User will be given an option to select the EV charging station based on the user wish as seen in figure 4.6. When user selected the EV station, the user will be redirected to another Web page and with performing necessary actions, the Total cost will be displayed for the power consumed by their EV's by performing all the necessary actions, the EV users are able to use the Main Interface Web Page. By performing all the necessary actions in the Main Interface Page, the Total cost for the Charging time period is displayed.

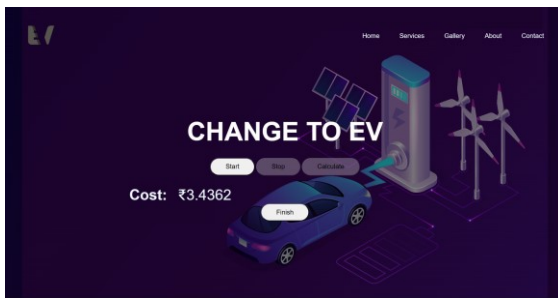


Figure 10. Displaying Cost

After getting the cost of power consumed by the EV's as shown in fig. 10, the EV users can finish the process by clicking the Finish Button. When the user clicks the Finish Button the user will redirect to the User Acknowledgement page.

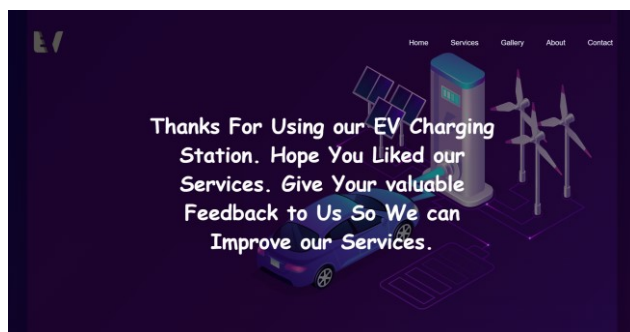


Figure 11. Acknowledgement Web Page for EV users

After getting the cost of power consumed by the EV's the user can finish the process by clicking the Finish Button. When the user clicks the Finish Button the user will redirect to the User Acknowledgement page. In fig. 11 acknowledgement Web page for the EV users. The proposed work is implemented with the methodology mentioned and the charging station is connected with the EV charger of an Electric Bike.



Figure 12. Connecting Prototype to the EV bike

In the fig. 12, the proposed EV charging station socket is connected to Hero Honda EV bike and power consumption cost is generated for the power consumed. The EV bike connected here is Hero Honda Photon bike, it has 1.82 kWh and it costs total 5 hours and 15 rupees for the full charge of this bike. With this EV station roughly of ₹4 is been estimated for an hour. So, for the 5 hours and for fully charger it costs 20 rupees which is 5 more rupees when comparing to manufacturers details.

The above calculations are only carried out without adding extra service costs such as maintenance cost, profit, government unit charges and many more. With adding all

this cost may varies. The overall charging may only vary up to ₹10 comparing to the original charging cost.

V. CONCLUSION

The proposed methodology has been implemented with the described parts and components. Each and every step is implemented and results are generated in each and every step for the better implementation of the proposed system. The methodology has been implemented using SCT-013-030 Non-invasive AC Current Sensor and integrated with Arduino Uno and Raspberry Pi which are major components used.

This proposed methodology will help the environment in many ways such as increasing a greater number of charging station which indirectly or directly increases the number of EV users. This ideology will overcome the one of the major drawbacks of the EV system that is users are unable to find the EV charging stations and problem in traveling more distance using EV's.

While coming to the improvement part, this can be integrated with security of OTP verification technology and a relay circuit which provides current only when OTP verification is done which provide more advance security since, currently a person needs to monitor while coming to the security of the device and accessing the charging port.

Hence the conclusion is that the proposed system is implemented with the discussed methodology to overcome the flaws of the EV mode of transportation. This proposed methodology is also eco-friendly and it aids to develop a side hustled business idea where Client provides Energy source as a service and the Customers are the consumers of that Energy sources

VI. REFERENCES

- [1] Shajahan, Altaf Hamed, and A. Anand. "Data acquisition and control using Arduino-Android platform: Smart plug." 2013 international conference on energy efficient technologies for sustainability. IEEE, 2013.
- [2] Juhana, Tutun, and Arif Indra Irawan. "Non-intrusive load monitoring using Bluetooth Low Energy." 2015 9th International Conference on Telecommunication Systems Services and Applications (TSSA). IEEE, 2015.
- [3] Ren, Sotara, et al. "A development of capacitive voltage sensor for non-intrusive energy meter." 2018 15th International Conference on Electrical
- [4] Princy, S. Emima, and K. Gerard Joe Nigel. "Implementation of cloud server for real time data storage using Raspberry Pi." 2015 Online International Conference on Green Engineering and Technologies (IC-GET). IEEE, 2015.
- [5] Gheorghe, A. C., E. Stan, and I. Udriou. "Electricity Consumption Measurement System Using ESP32." The Scientific Bulletin of Electrical Engineering Faculty 21.2 (2021): 23-26.
- [6] Thakare, Sanket, et al. "Implementation of an energy monitoring and control device based on IoT." 2016 IEEE Annual India Conference (INDICON). IEEE, 2016.
- [7] C. Yung-Chang, H. Wie-Hung, C. Shih-Hsien and T. C. Yu, "A Power Sensor Tag With Interference Reduction for Electricity Monitoring of Two-Wire Household Appliances," IEEE Xplore, vol. 61, no. 4, pp. 2062-2070, 2014.
- [8] D. Lawrence, J. S. Donnal, S. Leeb and Y. He, "NonThakare, Sanket, et al. "Implementation of an energy monitoring and control

- device based on-Contact Measurement of Line Voltage,” IEEE Xplore, vol. 16, no. 24, pp. 8990- 8997, December 2016
- [9] Villani, S. Benatti, D. Brunelli and L. Benini, “A contactless three phase autonomous power meter,” in 2016 IEEE SENSORS, 2016
 - [10] Bin Yu, Xu Lisheng, Yongxu Li. “Bluetooth Low Energy (BLE) Based Mobile Electrodiagram Monitoring,” 2012 International Conference on Information and Automation, ICIA, pp. 763-767, June 2012.
 - [11] Cepisca, C, Seritan, G, Cepisca, C,I, Grigorescu, S.D, - Digital Sampling Method in the Measurements of Electrical Power and Energy - In: Proceedings of the 9th WSEAS Int. Conf. on Mathematical Methods and Computational Techniques in Electrical Engineering, Book Series Title: Electrical and Computer Engineering, pp. 45-49, Arcachon, France, October 13-15, WOS000254019000008
 - [12] C. Villani, D. Balsamo, D. Brunelli and L. Benini, "Ultra-low power sensor for autonomous non-invasive voltage measurement in IoT solutions for energy efficiency," in Smart Sensors, Actuators, and MEMS VII; and Cyber Physical Systems, 2018
 - [13] Chimakurthi, Venkata Naga Satya Surendra. "An Optimal Cloud Based Electric Vehicle Charging System." *Asia Pacific Journal of Energy and Environment* 8.2 (2021): 29-38.
 - [14] Saqib, Mohd, et al. "Smart electric vehicle charging through cloud monitoring and management." *Technology and Economics of Smart Grids and Sustainable Energy* 2 (2017): 1-10.
 - [15] Septian, Belen, Peddapuram Rakesh, and S. Rajiv Dhora. "Iot based power monitoring system for diesel generator." *2020 5th IEEE International Conference on Recent Advances and Innovations in Engineering (ICRAIE)*. IEEE, 2020.